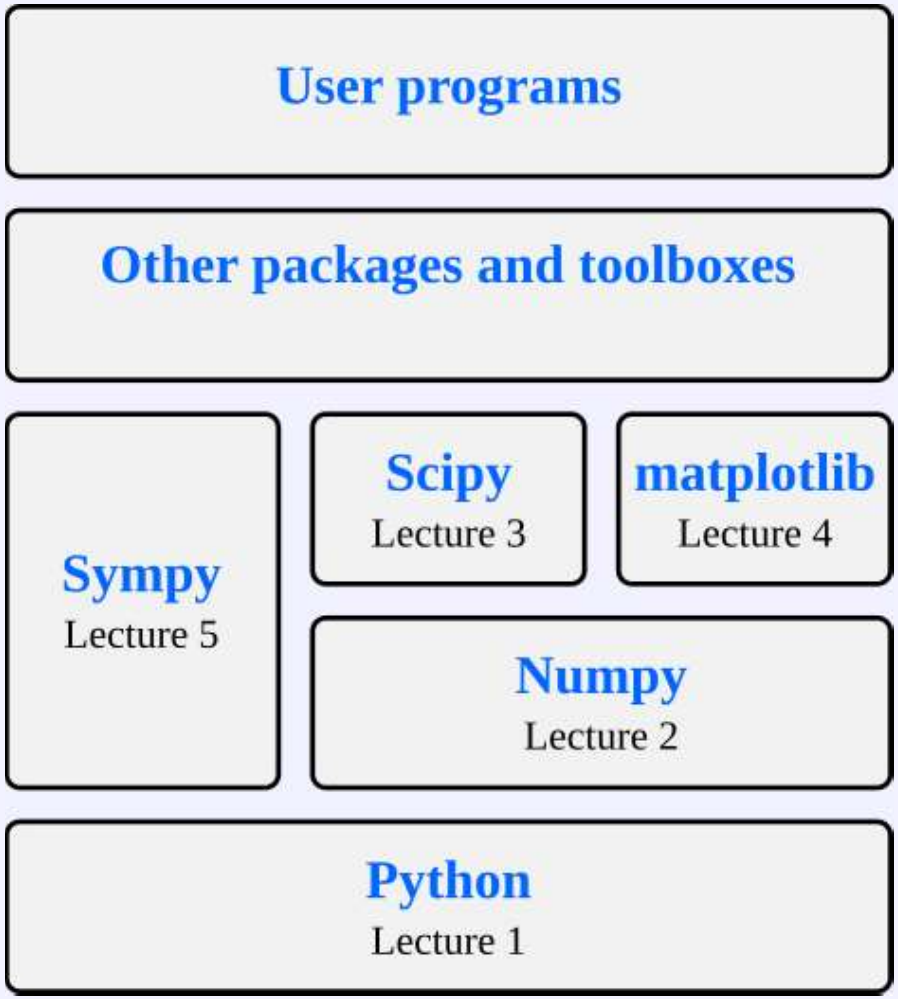
Partie III : Python, langage objet

Introduction

Pour le calcul scientifique



R. Johansson

- Contenu d'un cours (exemple)
- Objectif : atteindre une certaine maîtrise en M2 ...
- Plus de cours après le L3, projets uniquement.
- Apprentissage en autonomie
- S'investir en pratiquant
- Savoir trouver les informations (livres ou sites préférés)
- Acquérir des compétences qui feront la différence dans la recherche d'un emploi ou d'un stage.

Apprendre dans l'ordre, et de la meilleure manière

Python Crash-course

- installation, utilisation
- variable et type de données (data type) (Naming, ...)
- listes
- conditions if
- dictionnaire
- entrées et boucles (while, for)
- fonctions
- classes
- modules et paquets
- fichier and exceptions, tests

- Livres :
  - Une introduction à python-3, Cordeau, Pointa
  - Apprenez à programmer en Python (Le Goff), lire les jusqu'à la page 90 puis 162.
  - Apprendre à programmer en Python (Swinnen) (360 pages), page 1 à 134
  - Cours de python, Paris Diderot (sur Moodle)
- sites webs :
  - `https://python.developpez.com/cours/apprendre-python3/`
  - `https://courspython.com/`
  - `https://jrjohansson.github.io/computing.html`
  - `https://docs.python.org/3/`
  - `https://www.python.org/`

Pourquoi Python ? (R. Johansson)

It is a modern, general-purpose, object-oriented, high-level programming language

- General characteristics of Python :
  - clean and simple language : easy-to-read and intuitive code, easy-to-learn minimalistic syntax, maintainability scales well with size of projects.
  - expressive language : fewer lines of code, fewer bugs, easier to maintain.
- Technical details :
  - dynamically typed : no need to define the type of variables, function arguments or return types.
  - automatic memory management : no need to explicitly allocate and deallocate memory for variables and data arrays. No memory leak bugs.
  - interpreted : no need to compile the code. The Python interpreter reads and executes the python code directly.

Avantages (R. Johansson)

- The main advantage is ease of programming, minimizing the time required to develop, debug and maintain the code.
- Well designed language that encourage many good programming practices :
  - Modular and object-oriented programming,
    good system for packaging and re-use of code.
    This often results in more transparent, maintainable and bug-free code.
  - Documentation tightly integrated with the code.
- A large standard library, and a large collection of add-on packages.

Inconvénients (R. Johansson)

- Since Python is an interpreted and dynamically typed programming language,
  the execution of python code can be slow
  compared to compiled statically typed programming languages, such as C and Fortran.
- Somewhat decentralized, with different environment,
  packages and documentation spread out at different places.
  Can make it harder to get started.

Pour le calcul scientifique (R. Johansson)

- Python has a strong position in scientific computing
- Large community of users, easy to find help and documentation.
- Extensive ecosystem of scientific libraries and environments
- Great performance due to close integration with time-tested and highly optimized codes written in C and Fortran
- Good support for parallelism an GPU computing
- Readily available and suitable for use on high-performance computing clusters.
- No license costs, no unnecessary use of research budget.
- Intensément utilisé dans l'industrie et la recherche. (C.A.)

Programmation orientée objet (POO)

Tout est un objet en Python !

- concept de programmation qui permet de structurer des programmes très efficacement : force et succès de Python
- un **objet** est un conteneur défini par des **attributs** (variables) et des **méthodes** (fonctions) qui interagissent entre eux
- une classe est un moule qui permet de définir un type d'objet
- Une instanciation de classe est la création d'un objet de cette classe

objet "voiture"

- Attributs : couleur, type (course, tourisme, SUV), puissance, masse, nbre_passagers, ...,
- Méthodes : calcul de la vitesse, calcul de la distance de freinage, calcul de la consommation, affichage des caractéristiques, ajout d'une caractéristique, ...

La POO sera développée dans une séance de TP encadré, et appliquée dans les projets suivis.

Utilisation de Python : maîtriser les outils

Editeurs, Environnement de développements (IDE)

- Les IDE sont des éditeurs de texte améliorés qui permettent d'éditer du code source, de le corriger, et de l'exécuter dans un terminal.
- Les plus courants pour le python sont Pycharm, VSCode, Spyder
- On peut utiliser des éditeurs simples et exécuter le code source dans un terminal python mon_code.py

Jupyter Notebook en IPython

- On ouvre une console Python dans un éditeur HTML.
- On peut mélanger (alterner) des zones rédigées (texte, formules, images) et des zones de codes
- Très pratique pour monter des cours ou des TD / TP.
- Jupyter est installé automatiquement avec Anaconda.
- L'extension des fichiers IPython est iynb, pas "py".
- VSCode lit et exécute aussi ces fichiers.

33 mots réservés

- Apprendre les mots réservés :
- on ne les utilise pas en nom de variable ou de fonctions.
- Commandes de base du langage Python

import	To import a module	**as**	To create an alias
from	To import specific parts of a module		
class	To define a class	**del**	To delete an object
if	To make a conditional statement	**elif**	Used in conditional statements, same as else if
else	Used in conditional statements		
for	To create a for loop	**continue**	To continue to the next iteration of a loop
while	To create a while loop	**break**	To break out of a loop
in	To check if a value is present in a list, tuple, etc.	**is**	To test if two variables are equal
and	A logical operator	**not**	A logical operator
or	A logical operator		
def	To define a function	**return**	To exit a function and return a value
yield	To end a function, returns a generator	**pass**	A null statement, a statement that will do nothing
True	Boolean value, result of comparison operations	**False**	Boolean value, result of comparison operations
None	Represents a null value		
try	To make a try...except statement	**except**	Used with exceptions, what to do when an exception occurs
finally	Used with exceptions, a block of code that will be executed no matter if there is an exception or not	**raise**	To raise an exception
assert	For debugging		
with	Used to simplify exception handling		
nonlocal	To declare a non-local variable	**global**	To declare a global variable
lambda	To create an anonymous function		

Built-in functions → maison

- https://docs.python.org/3.10/library/functions.html
- Fonctions définies à la base de Python
- Leur emploi dépend du niveau de programmation et si POO.
- En Licence :
  - float, int, str, type, isinstance, complex
  - print, format, input
  - dict, list, tuple, set, len, reversed, all, any
  - enumerate, range, zip
  - max, min, open, pow, sum, round, abs
  - help
- liste complète pour Python 3.9 :

		Built-in Functions		
abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

Variables

Type de variables

- différents types : `int, string, boolean, float, complex, ...`
- passage d'un type à l'autre : `char(35), int(3), str(3.14), float("3.14"), ...`
- déterminer son type : `type(var), isreal(3.14), ...`

Naming : norme PEP8

- uniquement des lettres minuscules, chiffre et _, E : `young_modulus`
- constantes : lettres majuscules et _ : `PI`
- ne pas commencer par un chiffre
- pas de mots clés
- ni trop court, ni trop long mais expressif
- éviter le `l` et `O` qui se confondent souvent avec `1` et `0`
- penser que le nom doit être compréhensible pour les autres
- utiliser de préférence des mots en anglais sauf contre-indication

Le code fonctionnera même si vous ne respectez rien,
mais il ne sera pas professionnel et réutilisable, voir incompréhensible.

Chaînes de caractères

Séparateur d'instruction (sur une même ligne) : ;
L'opérateur == vérifie si l'égalité est vraie ou fausse.

- une chaîne de caractères est une liste de lettres :
  `x = "alphabet"; x[2] == "p"`
- Python offre de larges possibilités pour traiter des chaîne de caractères (cf TP)
- Il faut apprendre les formats : `"Mon nom est %s" % name`
- `"Mon nom est {:10s}".format(name)`
- `f"Mon nom est {name}"`
- `"Mon Nom Est PERSONNE".lower()`
- faire la différence : `"je m'appelle mr x"`, et `'je m appelle mr x'`
- pas de différence `"x"` et `'x'`
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Nombres

Pour les affectations, mettre un espace avant et après une virgule et le signe =

`var = value * 10`

- Faire la différence pour les opérations entre entiers et nombres réels.
- Par défaut les nombres sont réels
- Amusant : `1000, 1_000, 1e3` sont les mêmes nombres.
- Pour l'avoir en entier faire `int(1000)`
- Affectation multiples : `a, b = 10, 20` ou bien `a = 10; b = 20`
- Opérations simples sur les nombres à savoir.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Listes

Objet mutable, exemples

Une liste est un conteneur qui est composé d'un ensemble d'objets (nombres, caractères, listes, etc ...)

- `x = ["mot", 10, 3.14, [2, "autre mot"]]`
  `y = list(2, 4, 6, "mots", 3.14, []); y[0] = 3`
- L'index du premier élément est 0
- Beaucoup d'opérations sont possibles : ajout, retrait, sélection, recherche d'un élément, comptage, classement, ...
- Accessibilité : `x[1] == 10; x[3][1] == "autre mot"`
- `len(x) == 4`
- *slicing* (extraction de blocs) : `y[2:-1] == [6, "mot", 3.14"]`
- `y[-1] == []`, une liste vide.
- `[1, 2] * 2 == [1, 2, 1, 2]` (répétition)
- les listes sont **mutables** : on peut modifier son contenu
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Listes : attention danger !

En Python les objets sont définis par leur adresse en mémoire, pas par leur nom

Il y a, à cause de cela, de fréquentes erreurs dans la manipulation des listes

- `a = [10, 20]; b = a`
- `a[0]= 1`, la liste a et b sont identiques : `b == [1, 20]`
- Pour les séparer il faut faire une `deepcopy` de la liste a vers b.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Listes : manipulation

La méthode `.append()` permet d'ajouter des éléments à une liste

- `square = [x for x in [2, 5, 7]]`
- `prod = [x * y for x,y in zip([1,2], [-5,3.14])]`
- liste de nombres entiers : `x = [n for n in range(2,7,2)]` : on a `[2, 4, 6]`
- minimun, maximun, etc ...
- petit code à comprendre :

```
1 evennumbers = [ ]
2 for i in range(10):
3     evennumbers.append(2 * i)
```

Indentation obligatoire dans la boucle : 4 espaces recommendés.
Espace de part et d'autre d'un opérateur.

- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

tupple, n-uplets

Objet non mutable, exemples

Un tuple est un conteneur qui est composé d'un ensemble d'objets (nombres, caractères, listes, etc ...) qui ne peut pas être modifié.
Utile pour éviter des erreurs, ou dans le passage d'arguments dans les fonctions

- `x = ("mot", 10, 3.14, (2, "autre mot"))`
  `y = tuple(2, 4, 6, "mots", 3.14, []); y[0] = 3`
- Accession aux éléments comme pour une liste
- Toutes les méthodes des listes ne sont pas appliquables
- On apprend son utilité en codant.
- Mieux on connaît cet objet, plus on l'utilise.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

égalité booléenne, conditions, if

Exemples pour réfléchir

- `var_test == x <= 2; var_test == 1 < x < 3;`
  regarder les opérateurs booléens en Python
- Comprendre l'exemple :

```
x = 4
if  0 <= x <= 10:      # gestion des espaces
    print("ok")
elif x > 100:
    print("bof")
else:
    print("pas bon")
# en une ligne possible si une seule chose simple
if x == 4: print("x=4")
# tester si une liste n'est pas vide:
myliste = []
if myliste:
    print("not empty list)
```

Indentation obligatoire, ainsi que les deux points :

- On peut imbriquer les `if-then-else` mais fortement déconseillé au-delà de 2 rangs.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Test avec is et in

Les instructions `is`, `in` sont très employées :

- est-ce qu'une variable est "quelque chose" ?
- est-ce qu'un élément est dans une certaine liste ?

Comprendre :

```
x = ("bike", "car", "bus")
y = None
z = "auto"
if z in x:
    print(" %s dans ma liste" % z)
if y is None:
    print("la variable n'existe pas")
```

S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Les dictionnaires

dict

Les dictionnaires sont propres à Python : c'est une liste dont l'indexation se fait par mots clés.

- Très employé pour passer beaucoup de paramètres dans des fonctions
- Regarder quelques possibilités

```
dic0 = {"un": 100, "deux": [100, "cat"]} # new dict
dic1 = dict(un=200, deux= [10, "dog"])   # new dict
cond = dic0["un"] == 200       # False
print("the animal is %s " % dic1["deux"][1])  # "dog"
print("keywords: ", dic1.keys())        # keys method
print("keywords: ", dic1.values()) # values method
for key, value in dic0.items():         # items method
    print("keys: ", key, "\t value: ", value)
dic3 = {}
dic3["trois"] = 300  # new dict with a new key
```

- on peut avoir des dictionnaires dans des dictionnaires : l'indexation est double avec deux mots `u["x"]["y"]`
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Entrer une valeur et boucler

Entrer une valeur → maison

au clavier, toujours faire un message clair !

- L'utilisation de la fonctionnalité `input` est à proscrire.
- Analyser quelques possibilités :

```
k = int(input("enter an integer: "))
s = input("enter a string: ")
x = float(input("enter a real number: "))
x = float(input("enter a real number: \n > "))
```

- ajouter un espace après le : pour ne pas coller l'entrer à la demande
- `"\n > "` rajoute un "prompt" et l'entrée est sur une autre ligne.
- Pas plus d'une ou deux entrées au clavier, c'est vite lassant !
- Préférer les entrées directes `x = 2` dans un script Python.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Boucler

- la boucle est la base de l'informatique, regarder :

```
s = "mots"
# it is current to find (but take care):
for s in s:                    # recommended
    print("%s \n" % s)
for i in range(len(s)):        # not recommended
    print(s[i])
x = [1, 2, 3, 4]
for y, a in zip(x, s):         # super recommended
    print("letter : %s, integer %d" % (a, y))
for k, a in enumerate(s):
    print("letter : %s, rank %d" % (a, k))
```

- l'instruction `zip` permet de lire plusieurs listes à la fois, obligatoirement de même longueur
- `enumerate` pour lire la liste et avoir un compteur
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

While, break

- On ne sait pas quand on va sortir

```
s = "my name is no one"
cpt = 0
while s[cpt] != "s":
    cpt += 1
# or
for s in s:
    if s == "s":
        break
x = 1
while True:          # current: infinite loop
    x *= 2
    print(x)
    if x > 1000:
        break

```

- Pour les compteurs choisir des mots simples ("iter", "cpt") ou bien réserver les lettres "i, j, k, m, n".
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Utiliser des fonctions

Généralités

- L'emploi de fonctions est la base de la programmation.
- Choisir un nom simple et expressif, en lettres miniscules (PEP 8)
- Ne pas utiliser plus de 3 ou 4 arguments, sinon cela devient illisible (`dict`)
- Toujours mettre un commentaire explicatif (`docstring`)
- Une fonction ne retourne pas nécessairement une valeur ou bien `Return None`
- Une fonction doit faire un ou deux choses, et être courte (entre 15 et 20 lignes maxi)
- Indentation nécessaire au début de la fonction
- On peut créer des fonctions 'en ligne' : fonction `lambda`, à éviter sauf si nécessaire, cela arrive ...
- Attention à la portée des variables : locale, non locale ou globale
- Grouper les fonctions au début du script, ne pas mélanger avec le "main"

Exemple 1

```
A = 2      #  a constante
def square(x):
    """ set the x^2 """
    return x ** 2

def display_value(x):
    """ print on screen a value """
    print("value : ", x)

def multiply(x):
    """ return x * a """
    return x * A

def multiply2(x, a=1): # default value
    """ return x * a """
    return x * a

display_value(square(2))
print("double value  : %f ", multiply(4))
print("a x : ", multiply2(4, a=2)) # or multiply(4, 2)
print("default value", multiply2(4))
```

Mise en forme et fonction en ligne

```
1     def large_number_args(arg1=3,
2                           arg2=["mots", 2],
3                           arg3=True,
4                           arg4=dic(un=1, deux=2)
5                           )
6         """ return something with all the arguments """
7         pass                # do nothing here
8         return arg1, arg2, arg3, arg4
9     # lambda function:
10    f = lambda x: 2 * x          # to avoid if not necessary
11    print("inline function result: ", f(4))
12
```

- La mise en forme permet de bien mettre en évidence les différents arguments, avec des mots clés
- On peut faire encore mieux, voir `kwargs` (paramètres qu'on peut rajouter à la volée)
- Utiliser les fonctions `lambda` avec parcimonie et à bon escient
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Utiliser des classes

Notion de classe

- La **classe** est à la base de la programmation objet.
- C'est un conteneur, et donc une sorte de prototype qui contient des variables et des fonctions propres appelées **attributs** et **méthodes** que le développeur doit définir
- Comme tout objet, elle emporte avec elle des méthodes de bases communes à toutes les classes.
- Deux points de vue sont à comprendre et à développer.

Classe : point de vue du développeur

Le développeur doit être en mesure de créer une classe, des attributs et des méthodes afin de donner des capacités à l'objet créé.
Les points suivant sont à comprendre :

- l'objet `class(object)`
- les attributs ou méthodes spéciaux : `__init__`, `__str__`, `__repr__`, `__dict__`, `__slots__`, ...
- les attributs ou méthodes publiques ou privées
- les accesseurs (setteurs, getteurs)
- le résultat de `dir(MaClasse)`
- les notions de `staticmethod` et `classmethod` et de décorateurs
- la dérivation (inherance) à partir d'une ou plusieurs autres classes
- la surcharge
- le polymorphisme
- les classes abstraites

Classe : point de vue du de l'utilisateur

L'utilisateur doit être en mesure de comprendre et d'utiliser l'objet créé avec toutes ses capacités disponibles.
Les points suivant sont à comprendre :

- création d'une instance de classe
- récupération, affectation, modification et utilisation des attributs
- utilisation des méthodes
- définition ou lecture des valeurs des attributs par défaut
- utilisation de multiples instances
- gestion des imports

Exemple de création

Objet, conteneur, attributs, méthodes, instanciation

```
# -*- coding: utf-8 -*-
class Cars(object):
    """ Cars tool """
    def __init__(self, params):
        """ Class initialization """
        self.color = params["color"]
        self.model = params["model"]
        self.power = params["power"]  # in cv
        self.speed = None

    def set_speed(self):    # setters
        """ calculate speed"""
        self.speed = self.power * 736 / 250  # wrong formula

    def get_speed(self):    # getters
        """ return speed """
        return self.speed
```

Exemple d'utilisation

Objet, conteneur, attributs, méthodes, instanciation

```
r5 = Cars(dict(color="blue", model="citadine", power=50))
r5.set_speed()
print("speed in km/h  :", r5.get_speed(), r5.speed)
r5.color = "red"  # modify an attribut
r5.mass = 750     # add an attribut
print("new color      : ", r5.color)
f1 = Cars(dict(color="red", model="supercar", power=850))
print("power ratio    : ", f1.power / r5.power)
```

- instanciation, paramètres
- L'utilisation du point `.` est extrêment importante, elle fait appel à une méthode `instance.method()` ou attribut `instance.attribut`.
- apprendre à modifier un attribut, utiliser une méthode
- apprendre à ajouter un attribut
- méthodes génériques `__method__`, héritage mère - fille pour plus tard
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Modules

Généralités

- Le module permet de rassembler du code sur un thème donné pour le réutiliser
- Un module est un fichier ou bien un dossier contenant plusieurs fichiers, ou une arborescence
- Il existe des centaines de modules officiels en python
- Apprendre à créer ses propres modules
- On verra en TD quelques modules : numpy, matplotlib, os, string, ...
- Apprendre à installer un module (avec anaconda, Pycharm, pip)
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Import, from, as

```
import numpy as np
import matplotlib.pyplot as plt
from os import *    # everything is imported ...
from numpy import tan as tangente    # renaming an element
# Cars class is in ./my_firt_class/cars.py
from my_first_class.cars import Cars
#
x = np.linspace(0, 1, 101)
plt.figure()
plt.plot(x, np.cos(x))
print("tangente de 45 deg: ", tangente(np.deg2rad(45)))
#
r5 = Cars(dict(color="blue", model="citadine", power="50"))
```

- La commande `import` peut se trouver n'importe où dans le code
- Si le module importé contient des lignes de codes exécutables, elles s'exécutent. Attention !
- La commande `from ... import *` importe tout, c'est dangereux parfois.
- La commande `from ... import func1` permet de contrôler précisément ce qu'on importe.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Fichiers, entrées / sorties

Généralités

- Lire ou écrire un fichier (txt, csv, xls, jpg, png, svg, ...)
- Lire fichier avec des nombres dans une matrice : utiliser numpy `load`
- Lire ou écrire dans un fichier contenant du texte ou des nombres et du texte : utiliser `with open` avec la commande `readlines()` ou autre
- On peut aussi utiliser le module `panda` qui permet de traiter les données efficacement.
- Pour les sorties graphiques voir avec `matplotlib.pyplot.savefig`
- Pour trouver les ou un ensemble de fichiers au bonne endroit sur le disque, on peut utiliser des fonctions de type shell qui se trouvent dans le module `os`
- À voir au fur et à mesure ...
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Erreurs et exceptions

Généralités

- On essaie de faire quelque chose, et on code les possibilités suivant que cela fonctionne ou pas,
- On peut ajouter des messages d'erreurs dans des boucles ou autre pour gérer les **exceptions**, voir `raise`
- Très utilisé à l'ouverture d'un fichier (au cas où il n'existerait) pas ou pour éviter les divisions par 0, ...)
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

La gestion des exceptions ou des erreurs peut paraître accessoire, surtout pour un débutant, mais c'est en réalité très important :
c'est un critère de qualité d'un code en plus du suivi des normes PEP.

Exemple de try-except-else / raise

```
import numpy as np

x = float(input("enter a number :"))

# first possibility with raise
if abs(x) <= 1e-15 :     # astuce, ne jamais tester 0
    raise ValueError ("can not divide by 0!")

# second one
try:
    ans = 100 / x
except ZeroDivisionError:     # many error type exist
    print("can not divide by 0!")
    ans = np.nan                # Not a Number
finally:     # what happens, it is always performed
    print("value  100 / x : ", ans)
```

- Les deux premières possibilités vont faire stopper le programme avec le message d'erreur indiqué.
- Dans le dernier cas, le code continue de fonctionner.
- S'informer, regarder des exemples, comprendre, apprendre, pratiquer.

Programmation par tests : assert

C'est la méthode de programmation à privilégier pour les professionnels. Mais elle n'est pas enseignée en L3.

```
1
2  from numpy import inf
3
4  def inverse(x):
5      if x == 0:
6          return inf
7      else:
8          return 1 / x
9
10 # main
11
12 assert(inverse(1) == 1, "mauvais calcul dans fun")
13 assert(inverse(0) == inf, "mauvais calcul dans fun pour 1 / 0")
```

- La fonction `assert` renvoie toujours `True` ou `False`.
- Si l'assertion est fausse, le message s'affiche.
- Les assertions sont utilisées dans la phase de débugage, ensuite elles doivent être effacées du code source car elles ralentissent l'exécution.
- En général on créée un module spécial contenant uniquement les assertions lors du débugage, ce qui permet de conserver les tests pour de futurs développements.